Практическая работа №8. Работа с DataTable, DataSet и SqlDataAdapter

Цель работы

Научиться загружать данные SQL Server в объекты `DataTable` и `DataSet` через `SqlDataAdapter`, фильтровать и сортировать их в памяти, а также работать с несколькими таблицами и проверять локальные изменения строк.

Учебная база: `coffee_shop`. **Продолжительность:** 80–90 минут.

Задание

Часть 1. Подготовка проекта

Создайте консольное приложение C# или используйте проект предыдущих практик. Убедитесь, что установлен пакет `Microsoft.Data.SqlClient`.

Используйте настройку подключения `CoffeeShop` из `appsettings.json` предыдущего занятия. Если проект ещё не настроен, для локального учебного запуска допустима строка:

```
string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";
```

В начале файла подключите необходимые пространства имён:

```
using System.Data;
using Microsoft.Data.SqlClient;
```

Часть 2. Загрузка товаров в DataTable

Создайте метод `LoadProducts(string connectionString)`, который загружает товары и возвращает `DataTable`.

```
static DataTable LoadProducts(string connectionString)
{
    string sql = """
        SELECT id, name, category_id, price, is_available
        FROM products
        ORDER BY id
        """;

    using SqlConnection connection = new SqlConnection(connec
tionString);
    using SqlDataAdapter adapter = new SqlDataAdapter(sql, co
nnection);

    DataTable table = new DataTable("products");
    adapter.Fill(table);

    return table;
}
```

Вызовите метод и выведите количество строк:

```
DataTable products = LoadProducts(connectionString);
Console.WriteLine($"Загружено товаров: {products.Rows.Coun
t}");
```

Обратите внимание: метод `Fill()` синхронный. Соединение, переданное адаптеру в закрытом состоянии, может открываться адаптером на время загрузки автоматически.

Часть 3. Вывод каталога из памяти

Пройдите по строкам `products.Rows` и выведите:

- идентификатор;
- название;
- цену;
- доступность.

```
foreach (DataRow row in products.Rows)
{
    int id = row.Field<int>("id");
    string name = row.Field<string>("name")!;
    decimal price = row.Field<decimal>("price");
    bool available = row.Field<bool>("is_available");

    Console.WriteLine($"{id}. {name} — {price} руб. | " +
        (available ? "Доступен" : "Недоступен"));
}
```

После загрузки списка повторных SQL-запросов для вывода делать не нужно.

Часть 4. Фильтрация и сортировка

Выведите доступные товары по возрастанию цены с помощью `DataTable.Select()`.

```
DataRow[] availableProducts =
    products.Select("is_available = true", "price ASC");

foreach (DataRow row in availableProducts)
{
    Console.WriteLine($"{row["name"]} — {row["price"]} ру
б.");
}
```

Далее самостоятельно выполните:

1. Вывод товаров дороже 200 рублей.

2. Сортировку по цене от большей к меньшей.
3. Подсчёт количества доступных товаров.

Все три операции выполняйте **над загруженным** `DataTable`, без повторной загрузки из SQL Server.

Часть 5. DataSet с товарами и категориями

Создайте `DataSet` и загрузите в него две таблицы.

```
DataSet dataSet = new DataSet("coffee_shop");

using SqlConnection connection = new SqlConnection(connection
String);

using SqlDataAdapter productsAdapter = new SqlDataAdapter(
    "SELECT id, name, category_id, price, is_available FROM p
roducts",
    connection);

using SqlDataAdapter categoriesAdapter = new SqlDataAdapter(
    "SELECT id, name FROM categories",
    connection);

productsAdapter.Fill(dataSet, "products");
categoriesAdapter.Fill(dataSet, "categories");

Console.WriteLine($"Загружено таблиц: {dataSet.Tables.Coun
t}");
```

Выведите количество строк каждой таблицы. Проверьте, что `dataSet.Tables["products"]` и `dataSet.Tables["categories"]` доступны.

Часть 6. Связь между таблицами

Создайте связь категорий и товаров:

```
DataTable categories = dataSet.Tables["categories"]!;
DataTable productTable = dataSet.Tables["products"]!;

dataSet.Relations.Add(
    "category_products",
    categories.Columns["id"]!,
    productTable.Columns["category_id"]!);
```

Выведите названия категорий, а под каждой — названия соответствующих товаров:

```
foreach (DataRow category in categories.Rows)
{
    Console.WriteLine($"Категория: {category["name"]}");

    foreach (DataRow product in category.GetChildRows("catego
ry_products"))
    {
        Console.WriteLine($"  - {product["name"]}");
    }
}
```

Часть 7. Локальное редактирование

Выберите одну строку в таблице `products` и измените цену **только в памяти**:

```
if (products.Rows.Count > 0)
{
    DataRow firstProduct = products.Rows[0];

    Console.WriteLine($"До: {firstProduct["price"]}");
    firstProduct["price"] = 999m;

    Console.WriteLine($"После: {firstProduct["price"]}");
```

```
    Console.WriteLine($"Состояние: {firstProduct.RowState}");
}
```

Затем повторно проверьте цену того же товара в SQL Server. Она должна остаться прежней: метод `SqlDataAdapter.Update()` мы не вызывали.

Часть 8. Самостоятельная часть

Соберите консольное меню **«Каталог кофейни в памяти»**:

1. Загрузить данные о товарах и категориях из SQL Server.
2. Показать все товары.
3. Показать только доступные товары.
4. Отсортировать товары по цене.
5. Показать товары выбранной категории.
6. Изменить цену выбранного товара в памяти и вывести `RowState`.
7. Повторно загрузить таблицу из базы и сравнить цену с локальной версией.
8. Завершить программу.

Условия:

- используйте `SqlDataAdapter` для загрузки данных;
- храните наборы строк в `DataTable` и `DataSet`;
- после загрузки фильтрация и сортировка выполняются локально;
- проверяйте пустые результаты и некорректный пользовательский ввод;
- выводите понятные сообщения об ошибках;
- не изменяйте реальные цены в базе SQL Server.

Что необходимо продемонстрировать преподавателю

- Загрузку строк через `SqlDataAdapter.Fill()`.

- Просмотр данных `DataTable` без активного подключения.
- Фильтрацию и сортировку строк в памяти.
- Загрузку двух таблиц в `DataSet`.
- Работу отношения `DataRelation`.
- Локальное изменение строки и проверку `RowState`.
- Подтверждение, что изменения в памяти не записались в SQL Server.

Выводы

Письменно ответьте:

1. Чем `DataTable` отличается от `SqlDataReader`?
2. Для чего используется `SqlDataAdapter`?
3. В чём назначение `DataSet`?
4. Для чего нужен `DataRelation`?
5. Почему после локального изменения `DataRow` данные в SQL Server остались прежними?

Дополнительное задание

Добавьте в меню вывод количества товаров в каждой категории. Используйте загруженные таблицы и созданную связь `DataRelation`. Результаты отсортируйте по убыванию количества товаров. Обращения к SQL Server для вычисления статистики не выполняйте.